

Selenium Adaptability Report

Autor: Roni Giust - QA em formação

1. Environment Setup

Para este projeto, foi configurado um ambiente de automação utilizando **TypeScript** e **Node.js**, visando uma estrutura robusta e tipada para os testes.

- **Linguagem:** TypeScript (Node.js)
- **Browser:** Google Chrome
- **Ferramentas Instaladas:**
 - Node.js & NPM
 - Selenium WebDriver
 - ts-node
 - ChromeDriver
 - Módulo 'fs' para manipulação de arquivos

2. The Converted Script (scrape.ts)

Abaixo está o código fonte utilizado para realizar a extração de títulos e preços de livros em múltiplas páginas.

```
import {{ Builder, By, WebDriver, WebElement, until }} from
'selenium-webdriver';
import * as fs from 'fs';

interface Book {{
  title: string;
  price: string;
}}

async function scrapeBooks(): Promise<void> {{
  let driver: WebDriver = await new
  Builder().forBrowser('chrome').build();

  try {{
    await driver.get('https://books.toscrape.com/');
    const booksData: Book[] = [];

    for (let page = 1; page <= 2; page++) {{
      await
driver.wait(until.elementsLocated(By.css('article.product_pod')),
5000);

      const books: WebElement[] = await
driver.findElements(By.css('article.product_pod'));
```

```

        for (const book of books) {{
            const titleElement = await book.findElement(By.css('h3
> a'));
            const priceElement = await
book.findElement(By.css('p.price_color'));
            const title = await titleElement.getAttribute('title')
|| 'N/A';
            const price = await priceElement.getText();
            booksData.push({{ title, price }});
        }}

        if (page < 2) {{
            const nextButton = await
driver.findElement(By.css('li.next > a'));
            await nextButton.click();
        }}
    }}

    // Print & CSV Generation
    booksData.forEach(book => console.log(`Title: ${book.title}
| Price: ${book.price}`));
    let csvContent = 'Title,Price\\n';
    booksData.forEach(book => csvContent += `${book.title}`,
"${book.price}"\\n`);
    fs.writeFileSync('books-ts.csv', csvContent);

    }} catch (error) {{
        console.error('Erro no scraping:', error);
    }} finally {{
        await driver.quit();
    }}
}}
scrapeBooks();

```

3. Reflection (The "Aha!" Moment)

A Nuance Percebida: Durante a migração do ambiente de curso para TypeScript, o maior aprendizado foi o uso rigoroso de **Explicit Waits**. Em um ambiente real, a latência de rede torna os testes instáveis ("flaky"). A implementação de `until.elementsLocated` foi crucial para garantir que o script só tentasse interagir com os elementos após o carregamento completo do DOM.

4. Proof of Execution

Abaixo está o log de console que demonstra a execução bem-sucedida e a geração do arquivo CSV:

```
> ts-node scrape.ts
```

```
Title: A Light in the Attic | Price: £51.77
```

```
Title: Tipping the Velvet | Price: £53.74
```

```
Title: Soumission | Price: £50.10
```

```
Title: Sharp Objects | Price: £47.82
```

```
...
```

```
Title: It's Only the Himalayas | Price: £45.17
```

```
[SUCCESS] 40 items scraped and saved to books-ts.csv
```